MIS205 | W13 Cost Estimation Cheat Sheet

How to read this sheet: every box gives one cost *algorithm*, its formula in blue, and a short interpretation.

Red marks expensive options; **green** marks the typically efficient ones.

1. Notation

Table R

r number of records in R

b number of disk pages of R

bfr records per page $= \frac{r}{b}$

Index on attribute A

x_A height of the B$^+$ Tree

$bI1_A$ number of leaf pages

d_A distinct values of A

s_A selectivity (rows the σ returns)

Selectivity rules (estimating s)

Selectivity is the **number of rows** a σ is expected to return.

Predicate type	**Formula**	**Example on Product, $r = 100{,}000$**
Equality, key attribute	$s_A = 1$	`pNo = 'b30999'` $\rightarrow s_A = 1$
Equality, non-key attribute	$s_A \approx \frac{r}{d_A}$	`category = 'Book'` $(d_A = 100) \rightarrow s_A = 1{,}000$
Range ($A > v$ or $A < v$)	$s_A \approx \frac{r}{2}$	`unitPrice > 500` $\rightarrow s_A \approx 50{,}000$

2. SELECT cost – $\sigma_{A=a}(R)$

(SL) Sequential page scan

- If A is the key of R: $\frac{1+b}{2}$
- If A is not the key: b

Read every page; works without any index. **Worst case** when b is large.

(SI) Unclustered B$^+$ Tree

- If A is the key: $x_A + 1$
- If A is not the key:

$$x_A - 1 + \left\lceil bI1_A \cdot \frac{s_A}{r} \right\rceil + s_A$$

Descend the tree, then one random data-page read per matching row. Good when s_A is **small**.

(SIC) Clustered B$^+$ Tree

$$x_A + \left\lceil \frac{s_A}{bfr} \right\rceil$$

Descend the tree, then read **consecutive** data pages.

Best for range queries, which has sequential I/O.

Heuristic

Use **(SIC)** when the column has a clustered index *and* the predicate is a range.

Use **(SI)** for a highly selective equality.

Fall back to **(SL)** only when no helpful index exists.

3. PROJECT cost – $\pi_{A_1,\ldots,A_n}(R)$

3a. With duplicates allowed (plain `SELECT`)

(PL) Sequential scan

b

Read every page; project the wanted columns from each record.

(PI) Covering index

$x_A - 1 + bI1_A$

Index already holds every projected column. Scan only its leaves.

3b. Without duplicates (`SELECT DISTINCT`)

(PLS) Scan + sort

$b + b_{\pi_A} K \times \log_2 b_{\pi_A} + b_{\pi_A}$

Scan, sort the projected pages, then sweep to drop adjacent duplicates.

(PIND) Covering index

$x_A - 1 + bI1_A$

Leaves are already sorted on the index key. One pass emits distinct values.

Tip

b_{π_A} = pages occupied by **just the projected columns**, which is much smaller than b when the table is wide.

4. JOIN cost – $R \bowtie_{R.A=S.B} S$

Two tables are now in scope, so we **tag every table symbol with the table it describes**: b_R and b_S are page counts, r_R is the row count of R, bfr_S is records per page of S, and so on.

(JNL) Nested-loop join

$b_R + b_R \cdot b_S$

For every page of R, scan every page of S.

Quadratic: avoid for two large tables. Needs only 3 buffer pages, so it always works.

(JSM) Sort-merge join

$b_R + b_S$ (when both sides already sorted)

Sweep both inputs once in parallel. Linear in page count. Add the external-sort cost $K \cdot b_R \log b_R$ per unsorted side. **Best** when both inputs are already sorted (e.g., by a clustered index).

(JIL) Index-loop join (index on $S.B$)

- Unclustered: $b_R + r_R(x_B + s_B)$
- Clustered: $b_R + r_R\left(x_B + \left\lceil \frac{s_B}{bfr_S} \right\rceil\right)$

For every row of R, probe the index on $S.B$. **Best** when R is the smaller side and an index exists on the join key of S.

Worked comparison

$b_R = 1{,}000$, $b_S = 5{,}000$, $r_R = 50{,}000$, $x_B = 3$, $s_B = 1$, $bfr_S = 20$.

- (JNL): $1{,}000 + 1{,}000 \cdot 5{,}000 =$ **5,001,000** I/Os
- (JIL, unclustered): $1{,}000 + 50{,}000 \cdot 4 =$ **201,000** I/Os
- (JSM, sorted): $1{,}000 + 5{,}000 =$ **6,000** I/Os

Same join, **three orders of magnitude** difference.